# 1 What is FT

The Fourier Transform (FT) is a functional $F$ that brings real functions $h : \mathbf{R} \to \mathbf{R}$ into the transformed function $F(h) = H : \mathbf{R} \to \mathbf{R}$, this is done by means of this formula by definition:

$$H(\omega) = \int h(t)e^{-i\omega t}dt \tag{1}$$

To be noted that $H(\omega)$ are the spectral components of the function $h$, this means how much of the pulsation $\omega$ is in the spectrum of $h$. For example if $h(t) = \sin(\omega_0 t)$, $H(\omega)$ will be non zero only for $\omega = \omega_0$. Let us compute an example, let $h(t) = \cos(\omega_0 t)$, the transform now is:

$$H(\omega) = \int \cos(\omega_0 t)e^{-i\omega t}dt = \frac{1}{2}\int e^{-i(\omega+\omega_0)t} + e^{-i(\omega-\omega_0)t}dt = \frac{1}{2}(\delta(\omega+\omega_0) + \delta(\omega-\omega_0)) \tag{2}$$

Where we used the Dirac delta function.

# 2 Discrete FT

In real life real numbers sadly do not exists, (they are not real). Only integers exists in reality, (or rational numbers that are practically the same thing by a change of unit of measure), so we must deal with discrete signals. Imagine to sample at interval $\Delta$ a signal $h(t)$, so that we have a set of times $t_k$ and of values of the function $h_k = h(t_k)$. It is clear that if we sample too slowly, with a huge $\Delta$ we will get a very bad knowledge of the function. If the signal changes very fast we have to sample fast, this in formulae means that our sampling frequency that is $f_{max} = 1/(2\Delta)$ must be bigger of all the frequency of the signal, or at least bigger than the interesting ones. This is not a problem in reality, as there will be always a low pass filter in every system that gives a signal. So let us get $H(f)$ by making discrete the FT. First we define our "t-axes" and "f-axes", (remember $2\pi f = \omega$ in this notation), the times $t_k$ are $-N/2\Delta, (-N/2+1)\Delta, \ldots, (N/2-1)\Delta$, so they are $N$, and the frequency $f_n$ are $-N/2f_s, (-N/2+1)f_s, , (N/2-1)f_s$, with $f_s = \frac{1}{\Delta N}$.

$$H(f_n) = \int h(t)e^{-i2\pi f_n t}dt \simeq \Delta \sum_{k=0}^{N-1} h(t_k)e^{-i2\pi f_n t_k} \tag{3}$$

This can also be simplified by substituting the definition and translating the values of the times and frequencies because this formula is periodical of period $N$ in $n$ and $k$, i.e. now for example we have only positive values of $k$, the values from $k = 0$ to $k = N/2 - 1$ corresponds to the positive times, then from $k = N/2$ the $k$ are in correspondence with the negative times because the formula is invariant with respect to the substitution $k \to k + N$, and $N/2 - N = -N/2$. The same is done to the frequencies.

$$H(f_n) \simeq \Delta \sum_{k=0}^{N-1} h(t_k)e^{-i2\pi f_n t_k} = \Delta \sum_{k=0}^{N-1} h_k e^{-i2\pi nk/N} \tag{4}$$

This is the first important formula we must remember. pay attention to the fact that now the $f_n$ are ordered in the same way of the time, so $f_0 = 0, f_1 = 1fs, ...f_{N/2} = -N/2fs...f_{(N-1)} = -1fs$. Pay attention to this when implementing the algorithm. An example of algorithm that does this is `ft.c`:

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
#include<complex.h>

#define N 1024
#define T 1.

void fft(double *x, int dim){
 int k;
}

double f(double t){
 return exp(-t*t/T/T*100)*sin(2*M_PI*t*20);
}

double fill(double *x_t, double* t_t){
 double dt = 2*T/N;
 int i;
 for (i=0;i<N;i++){
  x_t[i] = f(-T + dt*i);
  t_t[i] = -T + dt*i;
 }
 return dt;
}

int main(){
 double x_t[N],t_t[N];
 double complex X_n[N];
 double f_n[N];
 int n,k;
 double complex sum;
 double dt=fill(x_t,t_t);
 double fs =  1./dt;
 for(n=0;n<N;n++){
  sum = 0;
  for(k=0;k<N;k++){
   sum+=x_t[k]*cexp(- 1.*2*M_PI*I*k*(-N/2+n)/N);
  }
  X_n[n]=dt*sum;
 }
 for(n=0;n<N;n++){
  f_n[n] = (-N/2+n)*fs/N;
  if(f_n[n] < 30. && f_n[n] > -30.){
   printf("%lf %lf\n",f_n[n],cabs(X_n[n]));}
 }
 return 0;
}
```

When you compile and rum it like this:

```
t@t:\~{}\$ gcc ft.c -lm ; ./a.out  | graph -T png | feh -
```

It gives the power spectrum of the function:

```
exp(-t*t/T/T*100)*sin(2*M_PI*t*20)
```

that is a sinusoid that has been made finite by a Gaussian decay. So we expect the same spectrum of the sine, two deltas slightly modified. In fact we see in output of this program:

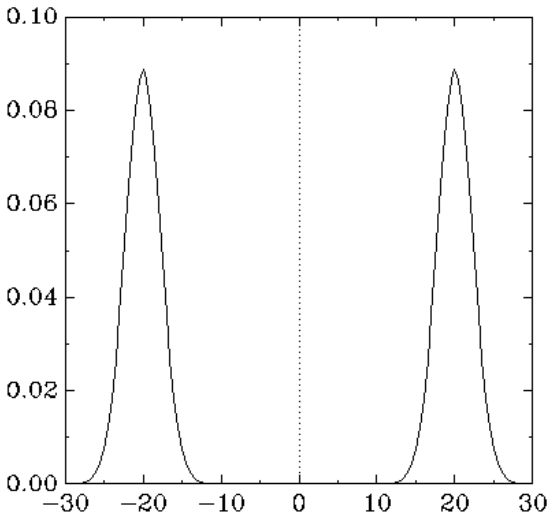


Figure 1: Output of ft.c.

This algorithm has a little problem, fixed $N$ that is how precise i want it to be the sampling, the complexity goes like $N^2$. We can fix this with the Fast FT (FFT).

# 3 The Fast FT, i.e. FFT

Now we have the problem of computing the $N$ sums of $N$ elements (dropping the $\Delta$ that is just a normalization factor):

$$H_n = \sum_{k=0}^{N-1} h_k e^{-i2\pi nk/N} \tag{5}$$

that is a $N^2$ problem, we can see that, defining as is usual $W = e^{2\pi i/N}$ we can split the sum over even and odd $k$:

$$H_n = \sum_{k=0}^{N-1} h_k W^{nk} = \sum_{j=0}^{N/2-1} h_{2j} W^{n2j} + \sum_{j=0}^{N/2-1} h_{(2j+1)} W^{n(2j+1)} = H_n^e + W^n H_n^o \tag{6}$$

Where we have defined the odd and even sub-sums. It is clear that now the problem is splitted in 2 $N/2$ problems. This can be repeated until (we assume $N$ a power of 2) we have a $N = 1$ transform that is trivial, i.e. the identity $H = h$. Then we can go back and reconstruct with $Nlog(N)$ operations the $H_n$. An algorithm that does such is `fft.c`:

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
#include<complex.h>

#define N 2048
#define T 1.


void fft(complex double* x, int dim){
 complex double *odd, *even, temp;
 int k;
 if(dim == 1) return;
 else {
  odd = ( complex double*) malloc( dim/2 * sizeof(complex double));
  even = (complex double*) malloc( dim/2 * sizeof(complex double));
  for(k=0;k<dim/2;k++){
   odd[k] = x[2*k+1];
   even[k] = x[2*k];
  }
  fft(odd,dim/2);fft(even,dim/2);
  for(k=0;k<dim/2;k++){
   temp = cexp(-2.*M_PI*I/dim*k);
   x[k]=even[k] + temp*odd[k];

   x[k+dim/2]=even[k] - temp*odd[k];
  }
 }
 free(odd); free(even);
 return;
}

double f(double t){
 return exp(-t*t/T/T*100)*sin(2*M_PI*t*20);

}
double fill(double complex *x_t, double* t_t){
 double dt = 2*T/N;
 int i;
```

```
 for (i=0;i<N;i++){
  x_t[i] = f(-T + dt*i);
  t_t[i] = -T + dt*i;
 }
 return dt;
}
int main(){
 double complex x_t[N];
 double t_t[N];
 double complex X_n[N];
 double f_n[N];
 int n,k;
 double complex sum;
 double dt=fill(x_t,t_t);
 double fs =  1./dt;
 for(n=0;n<N;n++){
  X_n[n] = x_t[n];
 }
 fft(X_n,N);
 for(n=0;n<N;n++){
  f_n[n] = (-N/2+n)*fs/N;
  if(n<N/2) f_n[n] = n*fs/N;
  if(n>N/2) f_n[n] = (-N + n)*fs/N;
  if(f_n[n] < 30. && f_n[n] > -30.){
   printf("%lf %lf\n",f_n[n],cabs(X_n[n]));}
 }
 return 0;
}
```

Running this program gives the same output of `ft.c`, but much faster. The algorithm can be improved by considering things that optimize the compute time by adapting the logic to what is best for the computer, things like bit reversal etc. But this is the basis of the FFT.